

# Übungsblatt 1

Punkte: 

|  |
|--|
|  |
|--|

 / 20

## Fortgeschrittene Algorithmen (Wintersemester 2022/23)

Bearbeitet von: Matthias Janßen

October 28, 2025

### Aufgabe 1

MaxProben(int[][] A, int m, int n)

```
//Merken aus welcher Richtung der Rover kommt
char previous[m][n]
//Matrix durchlaufen
for i = 1 to m do
    for j = 1 to n do
        //Startfeld braucht keine Aktion
        if(i == 1 & j == 1) then skip
        //Am linken Rand kann der vorherige Punkt nur oben ('0') liegen
        else if(i == 1) then
            //Die Summe des vorherigen und aktuellen Knoten wird gespeichert,
            //so wird der gesamte Wert des Weges gezählt
            A[i][j] = A[i][j] + A[i][j-1]
            previous[i][j] = '0'
        //Am oberen Rand kann der vorherige Punkt nur links ('L') liegen
        else if(j == 1) then
            A[i][j] = A[i][j] + A[i-1][j]
            previous[i][j] = 'L'
        else
            //Je nachdem ob die Summe des Weges von "oben" oder "links" und des aktu
            //wird der Vorgänger und Wert gespeichert
            max = A[i][j] + A[i-1][j]
            if A[i][j] + A[i][j-1] > max then
                max = A[i][j] + A[i][j-1]
                previous[i][j] = 'L'
            else
                previous[i][j] = '0'
            A[i][j] = max
//Pfad rekonstruieren indem von hinten nach vorne gegangen wird
path = []
while not (m == 1 & n == 1) do
    if previous[m][n] == 'L' then
```

```

    m = m - 1
    //'o' für Osten
    path.add('s')
else
    n = n - 1
    //'s' für Süden
    path.add('o')
return path.reverse, A

```

Der Algorithmus läuft in  $\mathcal{O}(m \cdot n)$  Zeit, da jedes Feld der Matrix genau einmal besucht wird.

## Aufgabe 2

a)



Figure 1: Auswahl des Algorithmus

Der Greedy Algorithmus aus Aufgabe a wählt die erste Aufgabe da sie die kürzeste ist. Dadurch kann nur eine Aufgabe gewählt werden, auch wenn es möglich wäre 2 Aufträge zu erledigen.

b)

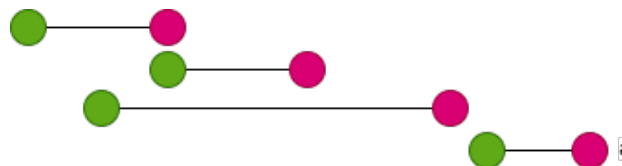


Figure 2: Auswahl des Algorithmus

Der Algorithmus wählt nur den letzten Auftrag a da dieser mit keinem anderem überlappt.

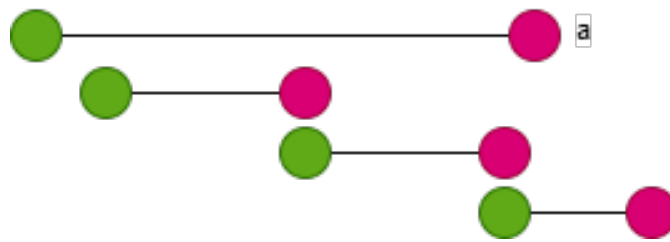


Figure 3: Auswahl des Algorithmus

c)

Der Algorithmus wählt Auftrag a da dieser den frühesten Startzeitpunkt hat. Dadurch kann nur ein Auftrag ausgewählt werden, obwohl 3 möglich wären.

### Aufgabe 3

a)

```
GreedyTimer(int[] t, int z)
    L = { }
    while z > 0
        for m = 0 to t.length do
            if t[m] > z
                L.add[t[m-1]]
                z = z - t[m-1]
                m = 0
            if m == t.length
                L.add[t[m]]
                z = z - t[m]
                m = 0
    return L
```

Der Algorithmus wählt immer die größte Taste aus die in die verbleibende Zeit passt. Da alle Tasten außer 1 den Teiler 5 Teilen wird so immer der optimale Weg gefunden.

b) Ein Pizzaofen mit den Tasten 1, 5 & 6. Für eine Eingabe von 10 Sekunden würde ein Greedy Algorithmus der immer die größtmögliche Taste wählt die Tastenkombination 6, 1, 1, 1, 1 auswählen anstatt 5, 5.